Open in app ↗

Towards Dev · Follow publication

io_uring: The Linux I/O Interface That Finally Makes Sense

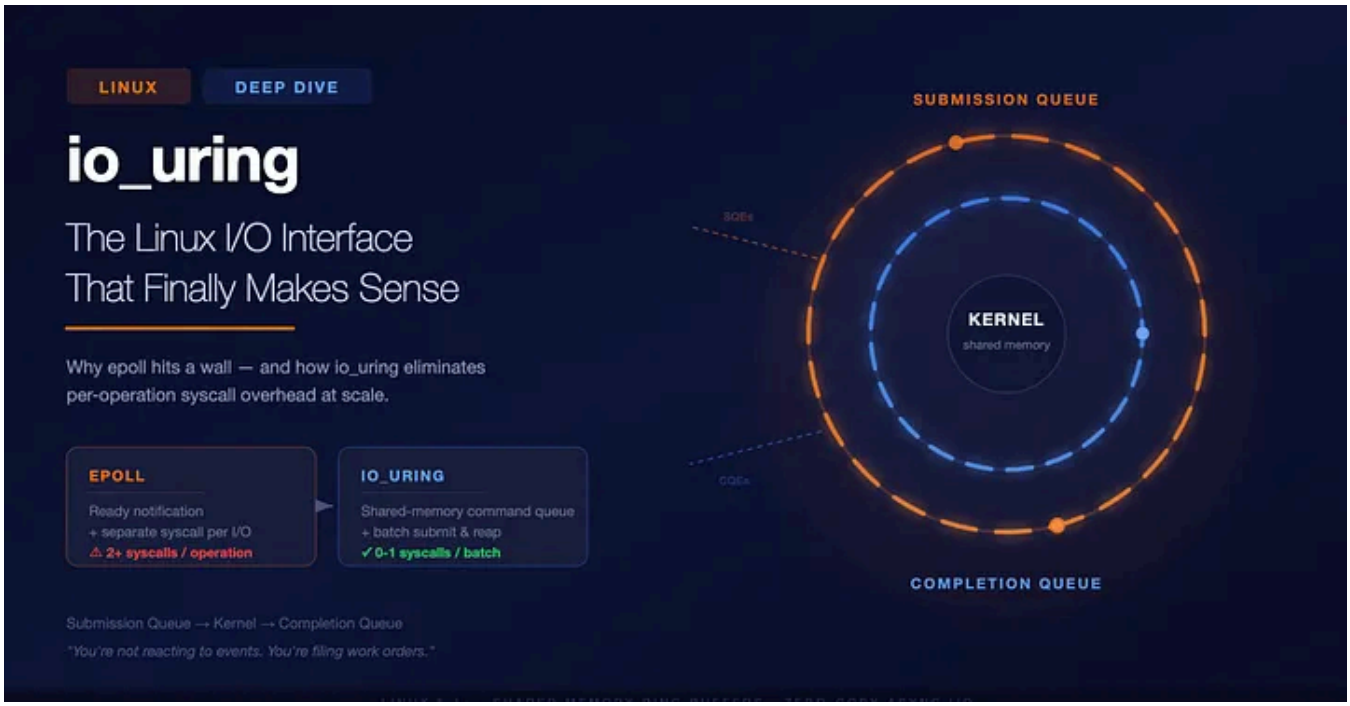
Why epoll hits a wall — and how io_uring eliminates per-operation syscall overhead at scale

8 min read · Apr 9, 2026

Sagar

Following

Listen Share ••• More



> Linux gave us better concurrency. io_uring gave us better economics.

What is the main problem with epoll ?

Every senior backend engineer has been here: you throw `epoll` at a concurrency problem, tune your thread pool, profile obsessively — and still hit a ceiling that shouldn't exist at your traffic numbers.

Here's the thing nobody says out loud:

"epoll tells you a file descriptor is ready. You still have to make a syscall to do the actual I/O. epoll scales concurrency. It does nothing about cost per operation."

You still pay a **syscall** for **every read. Every write. Every accept.** At 50k connections that's manageable. At 500k requests/sec, your CPU is spending a measurable slice of its life just crossing the user/kernel boundary — not doing your actual work.

```
Traditional epoll flow:

 Your App                Kernel
    |                       |
    |── epoll_wait() ──────►|  (blocks until fd ready)
    |◄── "fd 5 is ready"  ──|
    |                       |
    |── read(fd, buf) ─────►|  (syscall #2 — data copy)
    |◄── data  ─────────────|
    |                       |
    |── write(fd, buf) ────►|  (syscall #3)
    |◄── done  ─────────────|

   "Ready" notification + actual I/O = always 2+ syscalls minimum.
   At 500k req/sec, that's millions of kernel crossings. Per second.
```

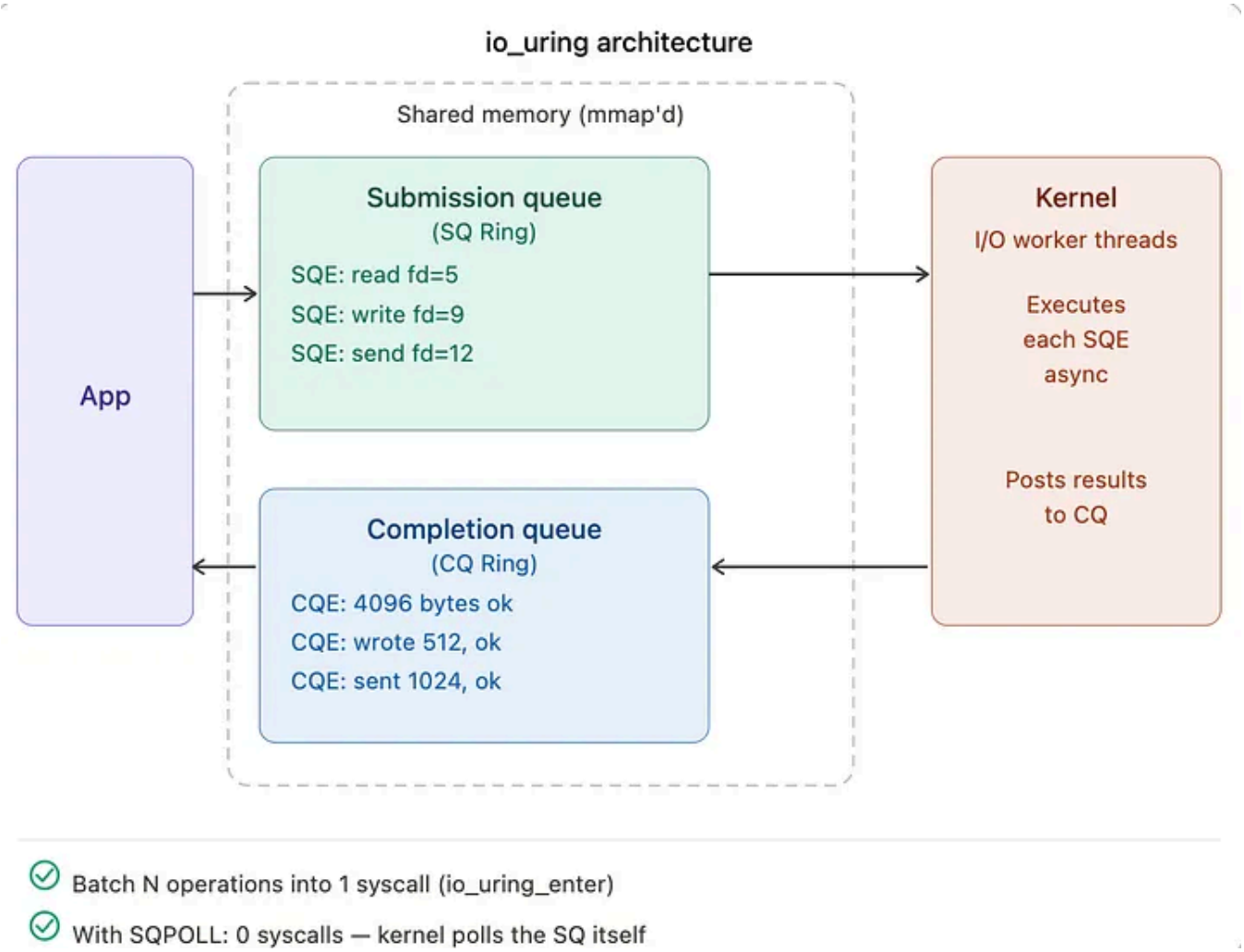
`select` and **`poll`** are worse — they degrade with fd count. **`epoll`** fixed that scaling problem in 2002. But the per-operation cost? Still there, untouched, for two more decades.

What is io_uring's big idea ?

Jens Axboe landed `io_uring` in Linux 5.1 (2019) with a different contract entirely:

> *Don't ask the kernel if you can do I/O. Tell the kernel what I/O you want. Pick up results when they're ready.*

The mechanism is two ring buffers in shared memory — no copying, no ownership handoff between user space and kernel:



In SQPOLL mode, a kernel thread watches the submission queue; otherwise, submissions are driven by system calls. You just write entries into shared memory. No `read()`, no `write()`. The kernel does its thing and drops results into the completion queue. You just read shared memory.

Note:
Completions can arrive out of order. Tag each SQE with user data so you know which finished.

The shift: you're not reacting to readiness events anymore. You're filing work orders and picking up completed jobs. The kernel can reorder, parallelize, and optimize the actual execution.

io_uring isn't just async I/O — it's a **shared-memory command queue between user space and the kernel.**

It's beautiful.

Usage — How to use `io_uring` in reality

Enough theory. Let's build a simple file reader using **`liburing`** (the sane way to use io_uring — nobody writes raw ring buffer code by hand). Raw `io_uring` setup is ~80 lines of `mmap` and struct wrangling.

Install liburing first:

```
# Ubuntu/Debian
sudo apt install liburing-dev

# Fedora
sudo dnf install liburing-devel

# Or build from source (always freshest)
git clone https://github.com/axboe/liburing.git
cd liburing && ./configure && make && sudo make install
```

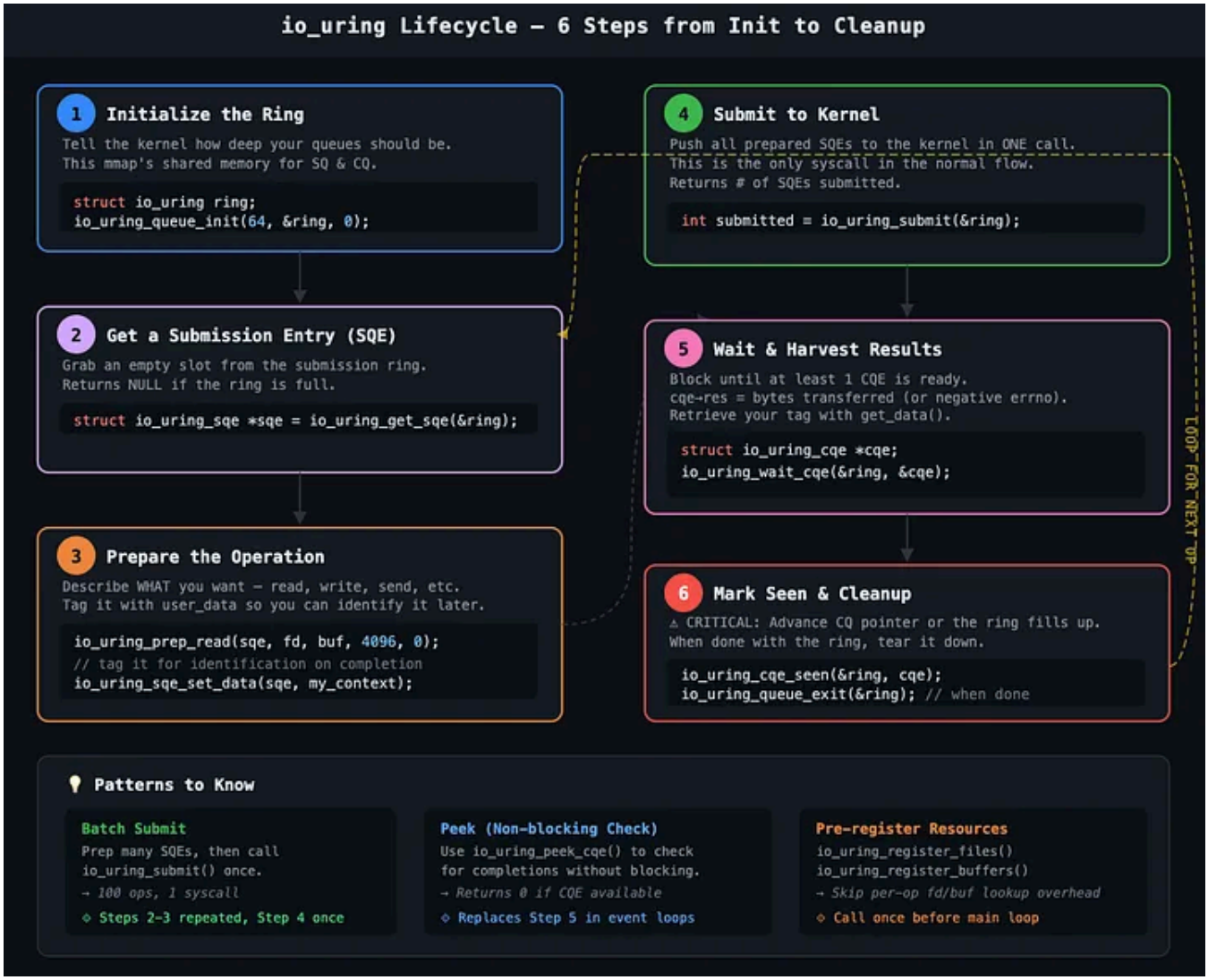
Basic file read with io_uring:

```
#include <stdio.h>
#include <fcntl.h>
#include <string.h>
#include <liburing.h>

#define BUFFER_SIZE 4096
#define QUEUE_DEPTH 1

int main() {
    struct io_uring ring;
    struct io_uring_sqe *sqe;   // Submission Queue Entry
    struct io_uring_cqe *cqe;   // Completion Queue Entry
    char buf[BUFFER_SIZE];
    int fd, ret;

    // 1. Spin up the ring — depth 1 means 1 in-flight op at a time
    ret = io_uring_queue_init(QUEUE_DEPTH, &ring, 0);
    if (ret < 0) {
```

```
        fprintf(stderr, "Queue init failed: %s\n", strerror(-ret));
        return 1;
    }

    // 2. Open file (still a regular syscall — totally fine here)
    fd = open("hello.txt", O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }

    // 3. Grab a free slot in the SQ
    sqe = io_uring_get_sqe(&ring);

    // 4. Describe the operation — read from fd into buf at offset 0
    io_uring_prep_read(sqe, fd, buf, BUFFER_SIZE, 0);

    // 5. Tag it — this comes back in cqe->user_data so you know what finished
    io_uring_sqe_set_data(sqe, buf);

    // 6. Submit — THE one syscall. Everything queued goes in one shot.
    ret = io_uring_submit(&ring);
    if (ret < 0) {
        fprintf(stderr, "Submit failed: %s\n", strerror(-ret));
        return 1;
    }

    // 7. Block until at least one CQE is available
    ret = io_uring_wait_cqe(&ring, &cqe);
    if (ret < 0) {
        fprintf(stderr, "Wait failed: %s\n", strerror(-ret));
        return 1;
    }

    // 8. cqe->res = bytes read on success, negative errno on failure
    if (cqe->res < 0) {
        fprintf(stderr, "Read error: %s\n", strerror(-cqe->res));
    } else {
        buf[cqe->res] = '\0';
        printf("Read %d bytes:\n%s\n", cqe->res, buf);
    }

    // 9. Tell the kernel you've consumed this CQE — advances the CQ head
    io_uring_cqe_seen(&ring, cqe);

    close(fd);
    io_uring_queue_exit(&ring);
    return 0;
}
```

Compile and run:

```
gcc -o io_uring_read io_uring_read.c -luring
echo "hello from io_uring" > /tmp/test.txt
./io_uring_read /tmp/test.txt
```



liburing API Cheat Sheet — Every Function You Need

RING SETUP & TEARDOWN — Called once at startup / shutdown

io_uring_queue_init(depth, &ring, flags)
Create a ring with **depth** SQ entries. Flags = 0 for defaults, or IORING_SETUP_SQPOLL etc.

io_uring_queue_init_params(depth, &ring, &params)
Same as above but with full **io_uring_params** struct for advanced config (SQPOLL idle, CQ size, etc).

io_uring_queue_exit(&ring)
Unmap shared memory, free resources. Call when completely done.

GETTING & SUBMITTING SQEs — Hot path — called per operation

io_uring_get_sqe(&ring)
Returns pointer to next empty SQE slot. Returns NULL if the SQ is full — submit first to drain.

io_uring_sqe_set_data(sqe, ptr)
Attach your own pointer/context to this SQE. You'll get it back from the CQE. **Don't skip this.**

io_uring_submit(&ring)
Flush all prepared SQEs to the kernel. Returns # submitted. This is 1 syscall for N ops.

PREP FUNCTIONS — Tell the SQE what kind of I/O to do — call one of these per SQE

File I/O
io_uring_prep_read(sqe, fd, buf, len, off)
io_uring_prep_write(sqe, fd, buf, len, off)
io_uring_prep_readv(sqe, fd, iovecs, nr, off)
io_uring_prep_writev(sqe, fd, iovecs, nr, off)
io_uring_prep_fsync(sqe, fd, flags)

Network I/O
io_uring_prep_accept(sqe, fd, addr, len, flags)
io_uring_prep_connect(sqe, fd, addr, len)
io_uring_prep_send(sqe, fd, buf, len, flags)
io_uring_prep_recv(sqe, fd, buf, len, flags)
io_uring_prep_close(sqe, fd)

Others: prep_openat · prep_statx · prep_splice · prep_timeout · prep_link_timeout · prep_cancel

HARVESTING COMPLETIONS — Reading results from the CQ ring

io_uring_wait_cqe(&ring, &cqe)
Block until 1+ CQE is available. **cqe→res** = return value (bytes or -errno).

io_uring_peek_cqe(&ring, &cqe)
Non-blocking check. Returns 0 if CQE ready, -EAGAIN if nothing yet. Great for event loops.

"Wait," you might be thinking, "that's MORE code than just calling `read()`. Why would I do this?"

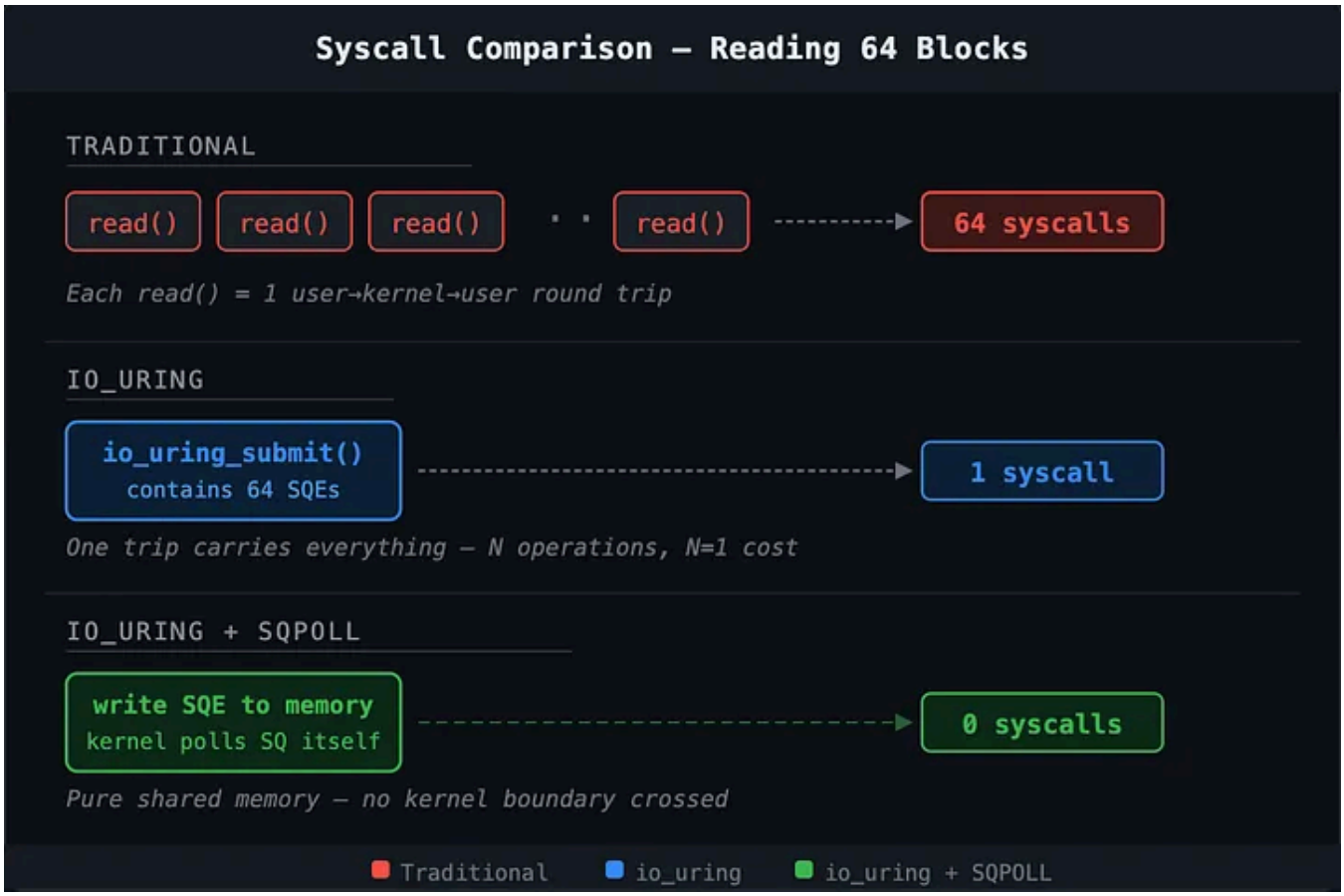
Fair point. For one read, you wouldn't. The magic happens at scale.

Where io_uring actually shines — batching

Single operations are nice. Batching is where the economics flip entirely.

Here's the real pattern. Say you need to read 64 chunks from a file. With traditional I/O that's 64 syscalls. With **io_uring**:

```
// Submit ALL 64 reads at once
for (int i = 0; i < 64; i++) {
    struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
    io_uring_prep_read(sqe, fd, buffers[i], BLOCK_SIZE,
                       i * BLOCK_SIZE);
    io_uring_sqe_set_data(sqe, &context[i]);
}

// ONE syscall to submit all 64
io_uring_submit(&ring);

// Reap completions (they might finish out of order!)
for (int i = 0; i < 64; i++) {
    struct io_uring_cqe *cqe;
    io_uring_wait_cqe(&ring, &cqe);
    // process...
    io_uring_cqe_seen(&ring, cqe);
}
```



Three Modes Worth Knowing

Mode	What It Does
Default	Normal async. One syscall per submit batch. Drop-in improvement over epoll for most workloads.
SQPOLL	Kernel thread polls SQ constantly. Zero syscalls on hot path. Burns a dedicated core. For: ultra-low latency, CPU to spare.
IOPOLL	Polling instead of IRQ for storage ops. For: NVMe drives, database write paths.

SQPOLL — removing the last syscall

Set `IORING_SETUP_SQPOLL` and the kernel spawns a dedicated thread that constantly watches the submission queue. You write SQEs into shared memory; the kernel picks them up without you ever calling `io_uring_submit()`.

```
struct io_uring_params params = {0};
params.flags = IORING_SETUP_SQPOLL;
params.sq_thread_idle = 2000;  // sleep after 2s of no work

struct io_uring ring;
int ret = io_uring_queue_init_params(
    QUEUE_DEPTH, &ring, &params
);
if (ret < 0) {
    fprintf(stderr, "sqpoll init: %s\n", strerror(-ret));
    return 1;
}
// From here on, just prep SQEs — kernel picks them up
```

> *"You batch work into shared memory, and a single syscall submits everything. With SQPOLL, even that syscall disappears from the hot path."*
>
> ***Trade-off:** SQPOLL burns a CPU core polling and requires `CAP_SYS_NICE` or root. It's meant for NVMe arrays and high-rate network stacks, not your TODO app.*

Comparison at a glance

	epoll + read/write	io_uring	io_uring + SQPOLL
Syscalls per op	≥ 2	~1 (amortized)	0
Batching	No	Yes	Yes
Min kernel	2.6	5.1 (5.10+ recommended)	5.11+
Complexity	Low	Medium	Medium–High
Op types	Readiness only	read, write, send, recv, accept, connect, fsync, openat, close...	Same
Best for	< 10k conn, simple servers	Databases, proxies, file servers	Extreme throughput, NVMe, DPDK-style net

Mistakes I made so you don't have to

1. Forgetting `io_uring_cqe_seen()`. The CQ is a fixed-size ring. If you don't advance the consumer pointer, it fills up and submissions silently stall. This one cost me two hours of staring at `strace` output.

2. Assuming in-order completion. CQEs can arrive in any order. Always tag your SQEs with `io_uring_sqe_set_data()` and identify results through `io_uring_cqe_get_data()`.

3. Ignoring `aligned_alloc` for direct I/O. If you open files with `O_DIRECT`, your buffers must be aligned (usually 4096 bytes). Regular `malloc` won't cut it. Use `aligned_alloc(4096, size)`.

4. Skipping fixed files/buffers. `io_uring_register_files()` and `io_uring_register_buffers()` let the kernel cache fd lookups and pin memory pages. For hot paths this can cut per-op overhead by 30-50%. Don't skip it.

5. Testing on old kernels. 5.1 had the basics. 5.6 added networking. 5.10 fixed edge cases. 5.15 stabilized everything. If you're debugging weird failures, check `uname -r` before anything else.

When to reach for io_uring (and when not to)

Reach for it when:

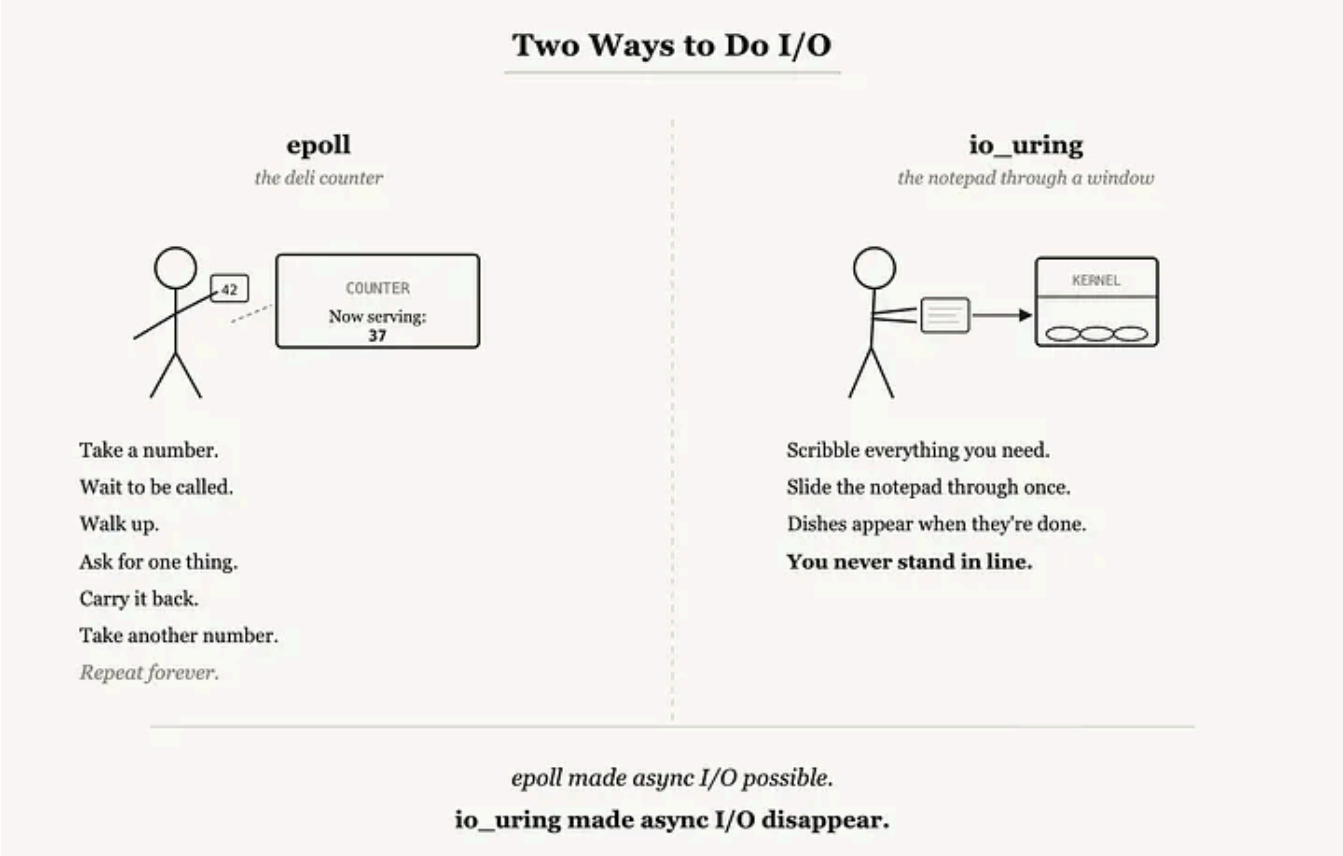
- Your flame graph shows syscall overhead in the top 5
- You're doing 10k+ concurrent I/O operations
- You need predictable tail latency (p99/p999)
- You're building a database engine, proxy, or storage service

Stick with epoll when:

- Low connection counts (< 1k)
- You need portability (macOS, BSDs don't have io_uring)
- Kernel upgrades aren't an option
- Simplicity outweighs raw throughput for your use case

Final Thoughts

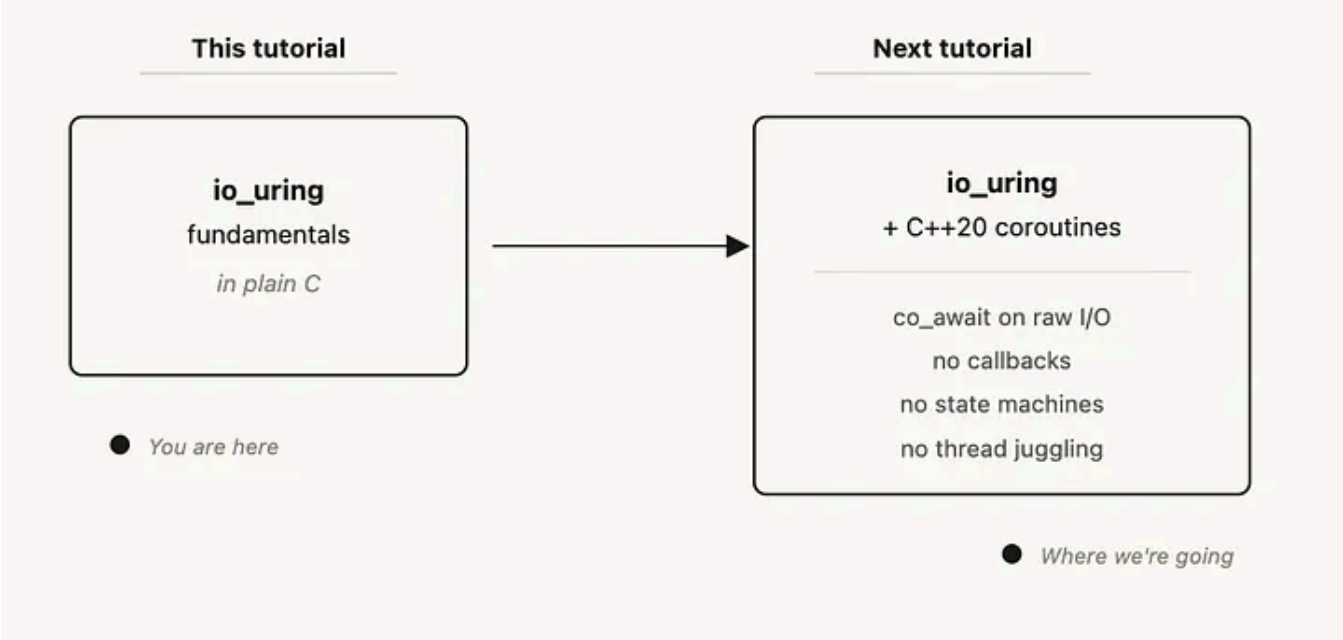
Here is a simple image with example that summerize the whole story:



One is a notification system — it tells you *something's ready* and waits for you to act. The other is a delegation system — you describe *everything you want* and the kernel brings it back finished. When your profiler stops showing `syscall` in the top 10, you'll know you picked the right one.

Whats next?

Here is the real reason why i wrote this post about **`io_uring`**.
Everything above was groundwork. We covered **io_uring** not as an end goal, but because it's the foundation layer for something much more powerful.



The idea is simple: every `io_uring_wait_cqe()` you saw in this tutorial becomes a `co_await` that suspends a coroutine, not a thread. Code that reads like blocking, runs like hand-rolled async.

```
// Sneak peek — next tutorial
int n = co_await ctx.async_read(fd, buf, 4096);
co_await ctx.async_write(fd, buf, n);
```

The hard part is behind you. The ring buffers, SQEs, CQEs — that's the machinery. Coroutines are just a nicer steering wheel.

Next up → *io_uring Meets C++20 Coroutines: Async I/O Without the Pain*

Found this article helpful? Please **Clap** 👏 and **follow** for more C++ and system programming content.

Programming | Software Development | Systems Programming | Linux | Linux Tutorial